

6. Write a program that reads a piece of delimiter symbols containing parentheses, braces and/or curly braces. The program determines whether its parentheses, braces, and curly braces are "balanced". The program can be done using stack data structure.

Hint: Process each character of the input.

- When a character is left delimiter, push it to stack.
- When a character is right delimiter, pop data from stack and check whether popped element matches right delimiter.
- It is balance when all matching are true and stack is empty when all characters have been processed.

Examples of balanced and unbalanced pieces of symbols:

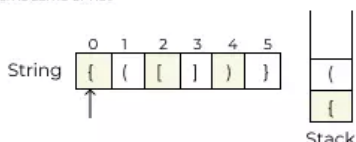
(({})) **Balanced!**
 { () } **Not balanced!**
 ((()) } **Not balanced**
 [()] **Balanced**

Check for balanced parentheses		
Expression	Balanced?	() or {} or []
) (	NO	
[()]	Yes	
[()]	NO	
[() ()]		
⇒ Last unclosed, first closed		

Balanced Parenthesis Problem

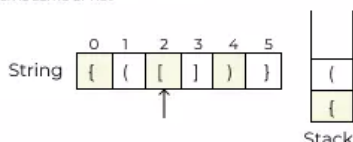
Step 1

Closing brackets Check top of the stack is same or not



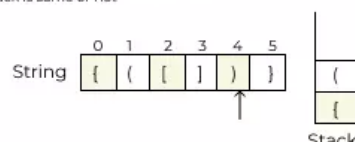
Step 3

Closing brackets Check top of the stack is same or not



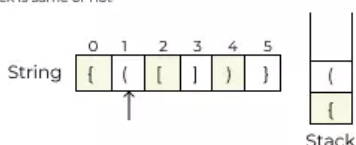
Step 5

Closing brackets Check top of the stack is same or not



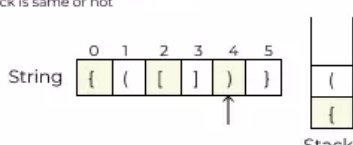
Step 2

Closing brackets Check top of the stack is same or not



Step 4

Closing brackets Check top of the stack is same or not



Step 6

Closing brackets Check top of the stack is same or not

